

ASSIGNMENT 7 — Page Object Model (POM) Restructure

1. Experiment Title

Page Object Model (POM) Restructure Using Selenium Python

2. Problem Statement

Take the working Selenium scripts developed in Tier 1 and restructure them using the **Page Object Model (POM)** design pattern. Create separate page classes containing locator definitions and UI interaction methods while keeping test assertions separate from the page classes.

3. Objective

- To implement the Page Object Model design pattern.
- To separate page locators from test logic.
- To create reusable page classes and UI methods.
- To improve the maintainability and organization of Selenium automation scripts.
- To keep test assertions separate from locators and page actions.

4. Tools, Software and Concepts Used

Tools & Software:

- Python
- Selenium WebDriver
- PyTest
- Google Chrome
- VS Code / PyCharm

Concepts:

- Page Object Model
- Page Classes
- Locators
- UI Methods

- Selenium WebDriver
- Test Assertions
- Code Reusability
- Separation of Test Logic and Page Logic

5. Source Code / Implementation Steps

Project Structure

Assignment_7_POM

├── login_page.py

└── test_assignment7.py

login_page.py

```
from selenium.webdriver.common.by import By
```

```
class LoginPage:
```

```
    USERNAME = (By.ID, "user-name")
```

```
    PASSWORD = (By.NAME, "password")
```

```
    LOGIN_BUTTON = (By.XPATH, "//input[@type='submit']")
```

```
    def __init__(self, driver):
```

```
        self.driver = driver
```

```
    def enter_username(self, username):
```

```
        self.driver.find_element(*self.USERNAME).send_keys(username)
```

```
    def enter_password(self, password):
```

```
        self.driver.find_element(*self.PASSWORD).send_keys(password)
```

```
    def click_login(self):
```

```
        self.driver.find_element(*self.LOGIN_BUTTON).click()
```

```
    def login(self, username, password):
```

```
        self.enter_username(username)
```

```
        self.enter_password(password)
```

```
self.click_login()
```

test_assignment7.py

```
from selenium import webdriver
```

```
from login_page import LoginPage
```

```
def test_login():
```

```
    driver = webdriver.Chrome()
```

```
    driver.maximize_window()
```

```
    driver.get("https://www.saucedemo.com/")
```

```
    login_page = LoginPage(driver)
```

```
    login_page.login("standard_user", "secret_sauce")
```

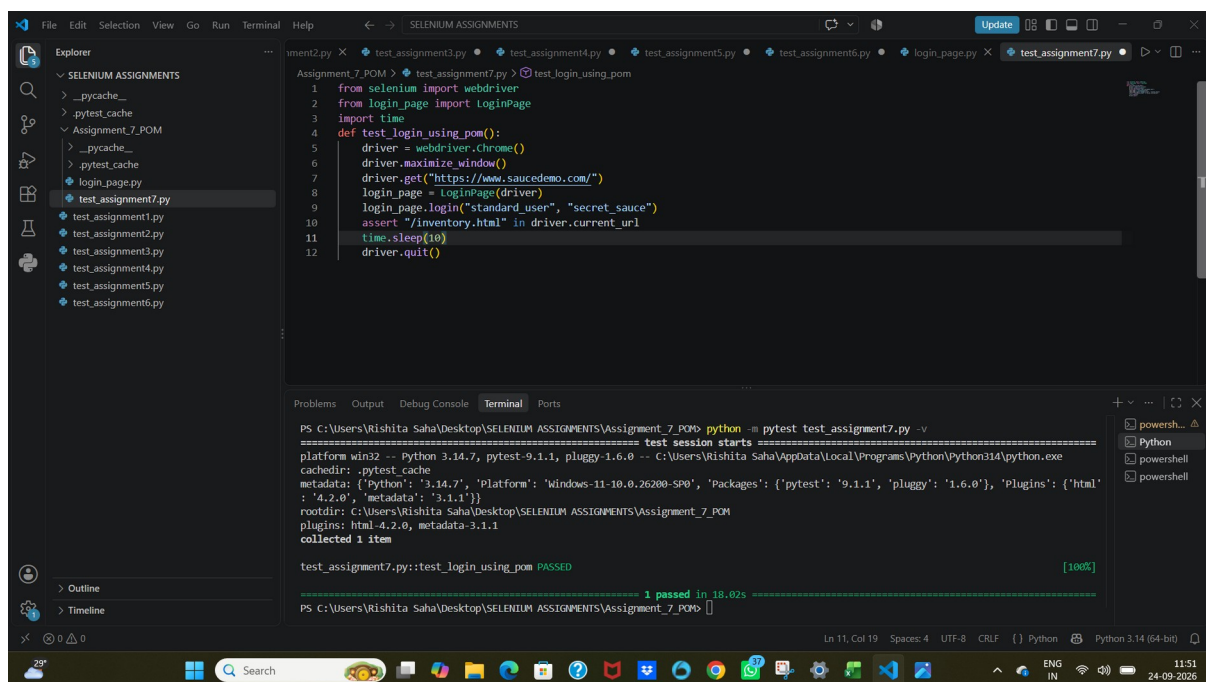
```
    assert "inventory.html" in driver.current_url
```

```
    driver.quit()
```

Execution

```
pytest -v
```

6. Output with Clearly Labelled Screenshots



The screenshot displays a Visual Studio Code editor window with a project named 'SELENIUM ASSIGNMENTS'. The file explorer on the left shows a directory structure with 'SELENIUM ASSIGNMENTS' at the top, followed by '._pycache_', '.pytest_cache', and 'Assignment_7_POM'. Under 'Assignment_7_POM', there are files for '._pycache_', '.pytest_cache', 'login_page.py', and 'test_assignment7.py'. The main editor window shows the code for 'test_login_using_pom.py', which includes imports for Selenium WebDriver, LoginPage, and time, followed by a test function that initializes a Chrome driver, maximizes the window, navigates to 'https://www.saucedemo.com/', logs in with 'standard_user' and 'secret_sauce', asserts the current URL contains 'inventory.html', sleeps for 10 seconds, and quits the driver. The bottom panel shows the 'Terminal' output, which includes the command 'python -m pytest test_assignment7.py -v' and the resulting output: 'platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Rishita Saha\AppData\Local\Programs\Python\Python314\python.exe', 'cachedir: .pytest_cache', 'metadata: {'python': '3.14.7', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0', 'Plugins': {'html': '4.2.0', 'metadata': '3.1.1'}}, 'rootdir: C:\Users\Rishita Saha\Desktop\SELENIUM ASSIGNMENTS\Assignment_7_POM', 'plugins: html-4.2.0, metadata-3.1.1', 'collected 1 item', 'test_assignment7.py::test_login_using_pom PASSED [100%]', and '===== 1 passed in 18.02s ====='. The status bar at the bottom indicates 'Ln 11, Col 19', 'Spaces: 4', 'UTF-8', 'CRLF', 'Python', and 'Python 3.14 (64-bit)'.

```
Assignment_7_POM > test_assignment7.py > test_login_using_pom
1 from selenium import webdriver
2 from login_page import LoginPage
3 import time
4 def test_login_using_pom():
5     driver = webdriver.Chrome()
6     driver.maximize_window()
7     driver.get("https://www.saucedemo.com/")
8     login_page = LoginPage(driver)
9     login_page.login("standard_user", "secret_sauce")
10    assert "inventory.html" in driver.current_url
11    time.sleep(10)
12    driver.quit()

PS C:\Users\Rishita Saha\Desktop\SELENIUM ASSIGNMENTS\Assignment_7_POM> python -m pytest test_assignment7.py -v
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Rishita Saha\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
metadata: {'python': '3.14.7', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0', 'Plugins': {'html': '4.2.0', 'metadata': '3.1.1'}}, 'rootdir: C:\Users\Rishita Saha\Desktop\SELENIUM ASSIGNMENTS\Assignment_7_POM', 'plugins: html-4.2.0, metadata-3.1.1', 'collected 1 item
test_assignment7.py::test_login_using_pom PASSED [100%]
===== 1 passed in 18.02s =====
PS C:\Users\Rishita Saha\Desktop\SELENIUM ASSIGNMENTS\Assignment_7_POM>
```

Test Execution Result

Show the terminal with the test passing.

7. Result

The Selenium login automation script was successfully restructured using the **Page Object Model**. Page locators and UI methods were separated from the test assertion, making the automation code more organized and reusable.

8. Observation

- Locators are maintained inside the page class.
- UI operations are implemented as reusable methods.
- Test assertions remain inside the test script.
- Changes to page locators can be managed centrally.

9. Conclusion

The experiment successfully demonstrated the **Page Object Model design pattern** for Selenium automation. Separating page-level operations from test logic improves the structure, reusability, and maintainability of the automation framework.